

TRIAGEM MÉDICA AUTOMÁTICA

Classificando urgência de laudos com pseudo-rotulagem BioBERT + modelo leve — End-to-End

Scikit-Learn

BioBERT

FastAPI

DVC

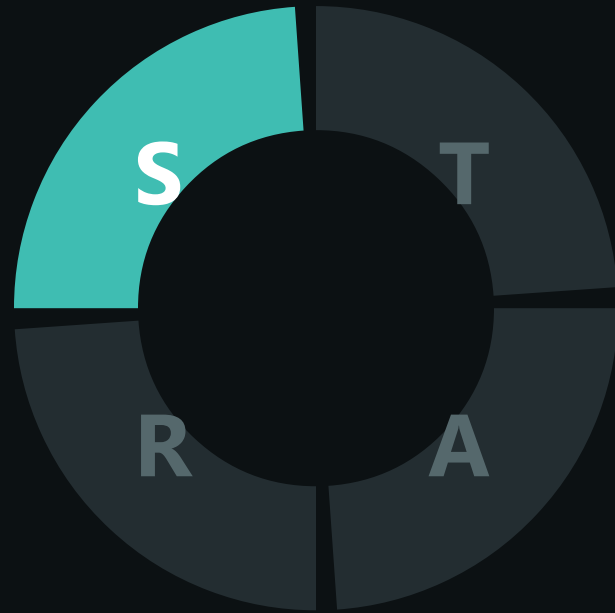
Airflow

Grafana

ONNX

M E D I C A L T R I A G E M L O P S

SITUAÇÃO · STAR : S



O Problema

Contexto

- Hospital precisa triar **laudos médicos em texto** por urgência — **urgente** · **atenção** · **normal** — no momento da chegada
- Requisitos da fase: API leve em Docker, **CI/CD**, orquestração com **Airflow**, **monitoramento** e otimização de latência
- O desafio central é de dados: o dataset público rotula **doenças**, **não urgência**

14.438

abstracts — Medical Abstracts TC Corpus

11.227

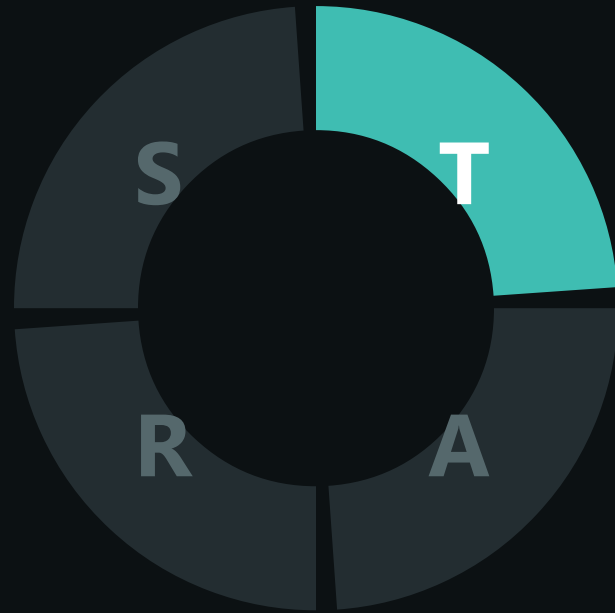
textos únicos após consolidação

5 → 3

categorias de doença no dataset → níveis de urgência exigidos

OBJETIVO: classificar a urgência de um laudo em < 1 ms — com recall alto na classe crítica

TAREFA · STAR : T



Nossa Solução

- 1 BioBERT pré-treinado (urgent / non-urgent) gera **pseudo-rótulos**
- 2 Regra de threshold **0,70** cria a zona de incerteza **atenção**
- 3 Split estratificado **70 / 15 / 15** — DVC, seed 42
- 4 **TF-IDF + Logistic Regression balanced** destila o conhecimento — 0,3 MB
- 5 FastAPI em Docker → **CI/CD no GitHub Actions** → Cloud Run
· Prometheus + Grafana
- 6 Retreino contínuo: **DAG Airflow sobre os stages DVC** +
quality gate + **dvc push**

Metas

Critério de modelo

Recall da classe urgente

Reprodutibilidade

dvc repro exato

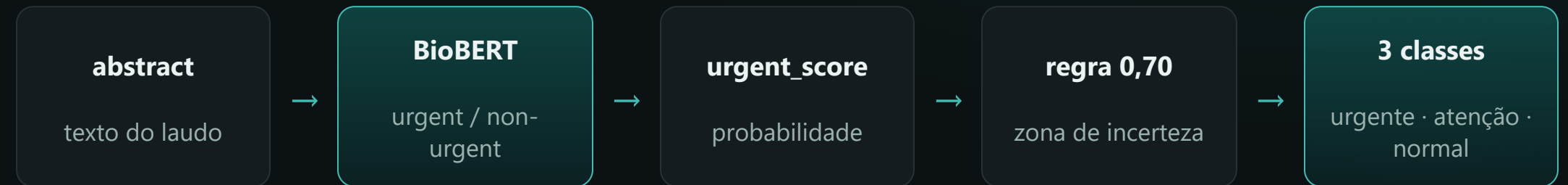
Otimização

Latência mensurável

AÇÃO · STAR : A



Pseudo-rotulagem — a Decisão de Dados

**11.227**

rotulados em ~1 min (GPU)

60% → 3%

truncamento (max_length 256 → 512)

5.000 · 4.873 ·**1.354**

normal · atenção · urgente

⚠ **O que a auditoria dos dados revelou:** a primeira rodada de rotulagem processou apenas os textos do início do alfabeto, e esse **viés de seleção** contaminava tudo que vinha depois. Resolvemos embaralhando o corpus e processando os 11 mil textos por completo. Vale lembrar também que a classe **atenção** não é um diagnóstico clínico: ela marca os casos em que o classificador ficou em dúvida e pede revisão humana.

AÇÃO · STAR : A



Do Baseline ao Modelo Final

Recall da classe urgente — conjunto de teste

MLP v1 · 720 amostras enviesadas		0,000
MLP v2 · corpus completo		0,591
LogReg balanced — final		0,813

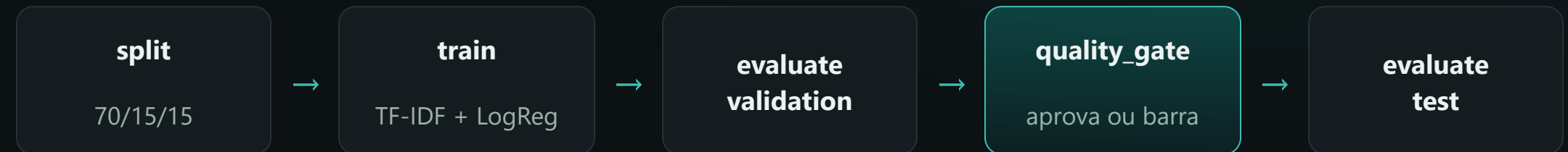
🏆 Por que o LogReg venceu

No macro F1 os dois modelos empatam em 0,75, mas a Regressão Logística consegue dar peso extra à classe minoritária com `class_weight='balanced'` — algo que o MLP do scikit-learn simplesmente não oferece. É isso que leva o recall de urgente de 59% para 81%. De quebra, o modelo é **25 vezes menor** (0,3 MB) e mais rápido, o que facilita o serving e a exportação para ONNX.

Numa triagem, o erro caro é deixar passar um caso grave — recall de urgente foi o critério de decisão

AÇÃO · STAR : A

Pipeline DVC + Quality Gate



O gate barra o deploy se...

- Macro F1 < **0,70** na validação
- Recall de **urgente** < **0,70**
- Alguma das 3 classes ausente das previsões
- Avaliado **só na validação** — o teste permanece intocado

Versionamento com DVC

- Dados e modelos em bucket **GCS** — o git só carrega código
- **dvc pull** materializa artefatos; **dvc repro** reproduz o pipeline
- Determinístico: seed 42, **uv.lock**, **37 testes**

Um modelo reprovado no gate nunca chega à avaliação final — nem à produção

AÇÃO · STAR : A



Produção — API FastAPI no Cloud Run

Medical Triage API 1.0.0 OAS 3.1

[/openapi.json](#)

API for real-time medical report triage. The model expects text in English.

default

GET `/health` Health**POST** `/predict` Predict

Schemas

HTTPValidationError > Expand all `object`**HealthResponse** > Expand all `object`**PredictionRequest** > Expand all `object`**PredictionResponse** > Expand all `object`**ValidationError** > Expand all `object`

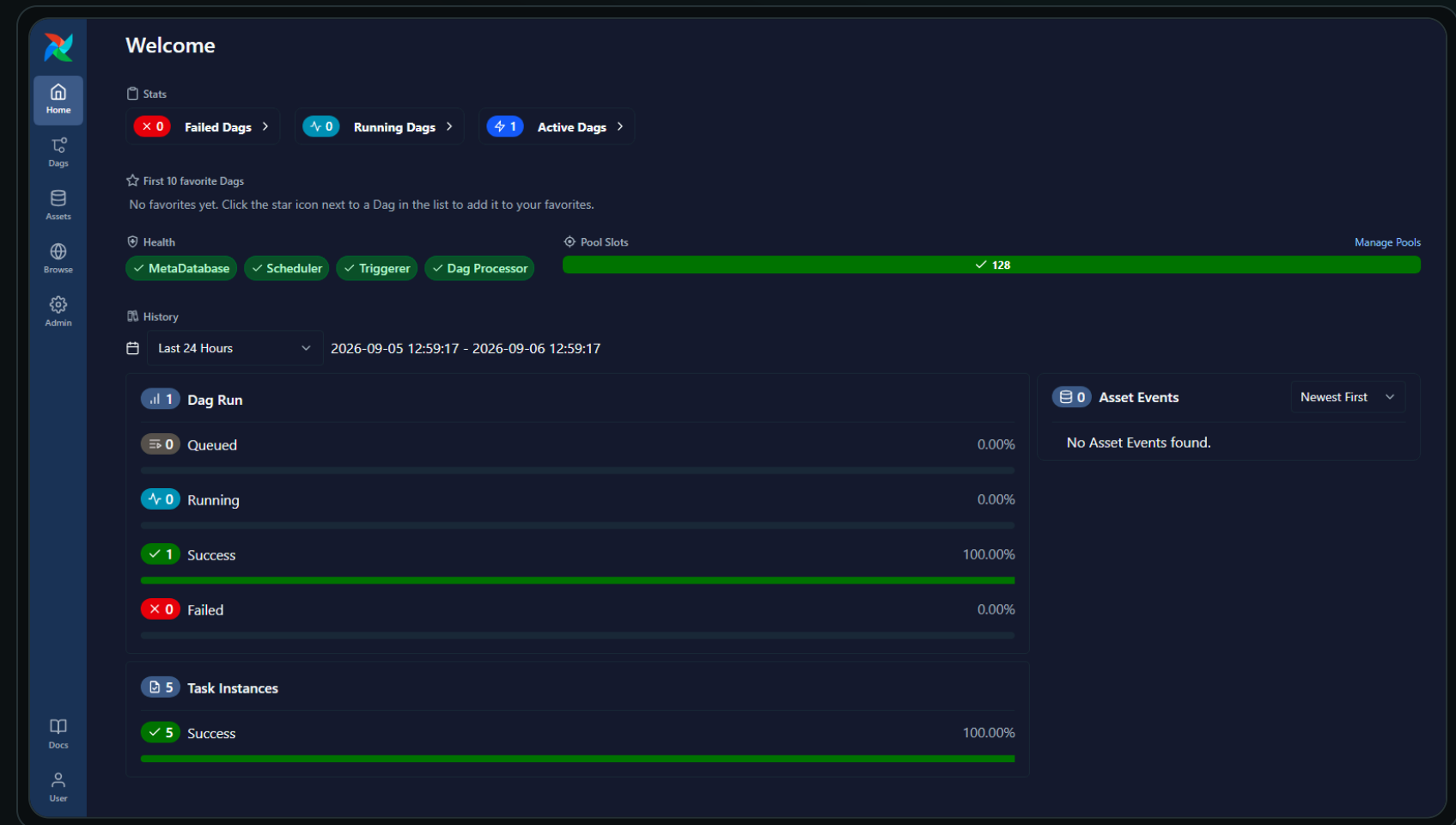
Serving

- **POST** `/predict` devolve `{triage_level, probabilities, model_version}`; **GET** `/health` é o probe do container
- Modelo carregado **uma vez no startup** — resposta em ~1 ms, sem torch nem GPU (imagem `python:3.11-slim`)
- **CD automatizado:** após o CI verde, o GitHub Actions faz `dvc pull` do modelo aprovado e builda a imagem pelo SHA do commit
- Publicação no **Cloud Run** como revisão candidata com smoke test, promoção a produção e rollback automático

AÇÃO · STAR : A



Continuous Training — DAG Airflow



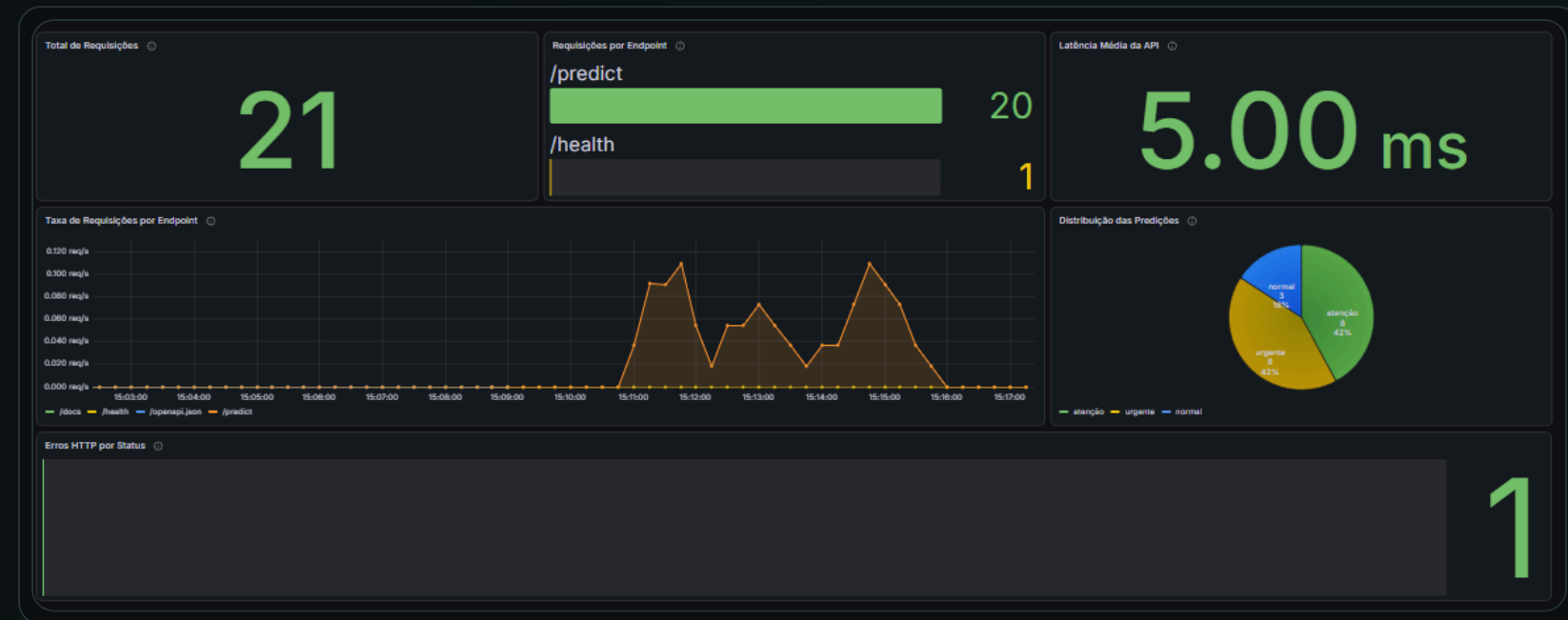
Retreino como código

- A DAG `medical_triage_training` é um wrapper dos **stages DVC** — a lógica vive só no `dvc.yaml`
- 7 tasks: `dvc pull` → `split` → `train` → `evaluate` → **gate** → `test` → `dvc push`
- O modelo aprovado vai para o bucket GCS, de onde o **CD** o pega para o deploy
- Batch **isolado** do caminho de inferência: o retreino nunca afeta a latência da API

AÇÃO · STAR : A



Observabilidade — Prometheus + Grafana



6 painéis provisionados

- Requisições, taxa por endpoint, latência média e erros HTTP
- Métricas `medical_triage_*` via prometheus-client; scrape da API local e do Cloud Run

Guard-rail de negócio

O painel de **predições por classe** existe por um motivo: o nosso primeiro baseline nunca previa `urgente`, e nenhuma métrica de infraestrutura teria denunciado isso. Se a série de urgentes zerar em produção, o problema está no **modelo**, não no serviço.

AÇÃO · STAR : A



Otimização de Latência — ONNX Runtime

Pipeline completo num único grafo — TF-IDF embutido

	SKLEARN (JOBLIB)	ONNX RUNTIME
Latência média	0,525 ms	0,079 ms
Latência P95	0,840 ms	0,118 ms
Tamanho do artefato	310 KB	208 KB

~6x

mais rápido na latência média (5,4–6,7x em 3 execuções)
inferência só com `onnxruntime`

✓ **E a otimização não muda o comportamento:** as previsões do ONNX coincidem com as do sklearn em 98,1% do conjunto de teste. Nos 32 casos em que discordam, as duas classes mais prováveis estavam praticamente empatadas (diferença média de 0,019) — são **empates técnicos**, não erros, e ficaram documentados no benchmark.

RESULTADO · STAR : R



O que Entregamos

0,768

Accuracy (teste)

0,750

Macro F1

0,813**Recall urgente** — vs 0,000
do baseline**0,08 ms**

latência média (ONNX)

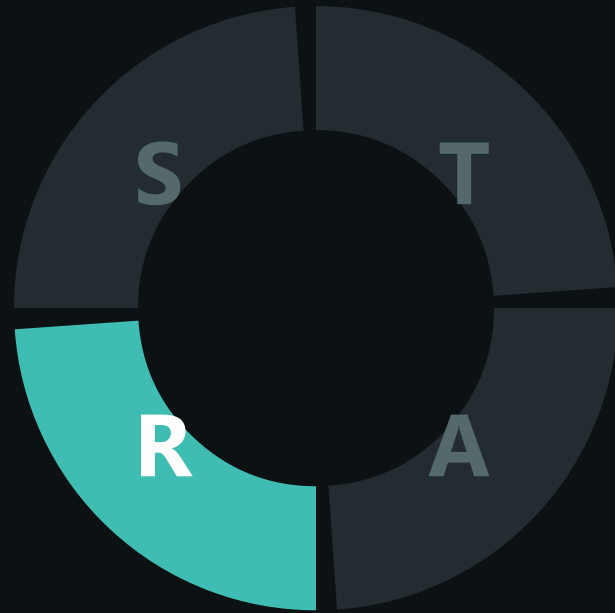
Requisitos do challenge — todos cumpridos

- Modelo NLP leve funcional ☒
- DAG Airflow funcional ☒
- API FastAPI + Dockerfile ☒
- Compose + Grafana (6 painéis) ☒
- CI/CD GitHub Actions ☒
- Otimização ONNX + comparativo ☒

Verificável

- API pública no **Cloud Run** — deploy automático pelo **CD** a cada CI verde
- CI + CD verdes · stack completa em 2 `docker compose`
- `dvc pull` + `dvc repro` reproduzem tudo

RESULTADO · STAR : R



Conclusões e Próximos Passos

🧠 Aprendizados-chave

- **Pseudo-rotulagem exige auditoria.** O viés que zerava o recall de urgente só ficou visível quando processamos o corpus inteiro — se tivéssemos confiado na amostra parcial, o problema teria ido para produção.
- **Um quality gate automatizado vale mais que revisar métricas na mão.** O pipeline barra sozinho qualquer modelo abaixo do mínimo, sem depender de alguém olhar o número certo na hora certa.
- **Separar os ambientes simplifica tudo.** A API não carrega torch e o treino não carrega FastAPI; a imagem de serving fica leve e o deploy, previsível.
- **Baselines fortes são o teste de honestidade.** Só dá para dizer que o modelo final é bom porque medimos exatamente quanto ele ganha de cada alternativa.

✅ Entregues

- Modelo + API + CI + DAG + monitoramento + ONNX
- Documentação completa e reprodução por DVC

📌 Próximo passo

- Validação clínica do threshold com especialistas
- Alertas Grafana sobre o guard-rail de **urgente**